



ENSEA

Beyond Engineering

SYSTÈMES À MICROCONTRÔLEUR

Compte Rendu TP4 –
Des usages multiples des Timers

1. Préparation

En allant dans l'onglet « Clock configuration » du fichier .ioc, on note que les deux « timer clocks » APB1 et APB2 sont réglés à $F_{APB} = 80MHz$.

En regardant ensuite la configuration de TIM4 (présent dans la section « Timers » de l'onglet « Pinout & Configuration »), on peut voir les valeurs écrites dans les registres Prescaler et AutoReload Register du timer. On note ici $PSC = 0$ et $ARR = 65535$.

On peut alors calculer la fréquence f_{T3} alimentant le compteur du Timer 3 :

$$f_{T3} = \frac{F_{APB}}{(PSC+1)} \underline{\text{AN:}} f_{T3} = \frac{80 \cdot 10^6}{(0+1)} = 80 \cdot 10^6 Hz$$

On a 400 pas/tour, la fréquence de pas nécessaire pour avoir 2 tours par seconde est donc $f_{Step} = 400 \times 2 = 800Hz$

On a alors :

$$ARR = \frac{f_{T3}}{f_{Step}} \quad \underline{\text{AN:}} f_{Step} = \frac{80 \cdot 10^6}{800} = 100 \cdot 10^3$$

On peut ensuite remplir le tableau donné en énoncé pour $PSC = 79$:

Vitesse (tr/min)	Vitesse (tr/s)	ARR
1	0,0167	149700,599
2	0,033	75757,5758
5	0,083	30120,4819
10	0,167	14970,0599
30	0,5	5000
60	1	2500
90	1,5	1666,66667
120	2	1250

Puisque ARR est renseigné sur 16 bits, sa valeur ne peut pas dépasser 65535, ce qui n'est pas compatible avec les valeurs calculées précédemment. Pour pallier à ce problème, on modifie la valeur de *PSC* à 799 et on obtient à présent les valeurs suivantes qu'il est possible d'écrire sur 16 bits :

Vitesse (tr/min)	Vitesse (tr/s)	ARR
1	0,0167	14970,05988
2	0,033	7575,757576
5	0,083	3012,048193
10	0,167	1497,005988
30	0,5	500
60	1	250
90	1,5	166,6666667
120	2	125

2. Partie I : le driver du codeur incrémental

On teste tout d'abord le bon fonctionnement de l'écran oled et de la fonction `printf`



Fig. 1 et 2 – tests réussis de la fonction `printf` et de l'affichage sur l'écran

On vient ensuite créer dans le projet le fichier `encoder.c` et son entête associée. On ajoute dans cette dernière la définition de la structure `ENCODER` et les deux prototypes de fonctions donnés dans l'énoncé. La variable `encoder` doit quant à elle être déclarée dans le fichier de code `.c`, en effet si elle est déclarée dans le fichier d'entête et que celui-ci est inclus dans plusieurs autres fichiers de code, alors la variable serait déclarée plusieurs fois. Il n'y aurait pas d'erreur puisque le terme `static` permet de limiter la portée de la variable au fichier `.c` dans lequel elle est déclarée, mais on aurait tout de même une consommation inutile de mémoire.

On relève ensuite la configuration du Timer 2. `TIM2` est configuré en interface encodeur sur `PA0` et `PA1`, et alimenté par l'horloge `APB1` à 80MHz .

Le mode "encodeur" du timer décode deux signaux en quadrature (ici $A = TI1$ et $B = TI2$) et met à jour le registre `CNT` à chaque transition, en comptant vers le haut ou vers le bas selon lequel des deux signaux est en avance en phase.

On peut ensuite coder les fonctions **encoder_init** et **encoder_read** :

```
void encoder_init(TIM_HandleTypeDef * htim_param, int32_t min, int32_t max){
    encoder.htim = htim_param;
    encoder.min_value = min;
    encoder.max_value = max;
    HAL_TIM_Encoder_Start(encoder.htim, TIM_CHANNEL_ALL);
    encoder.htim->Instance->CNT=0;
}

int32_t encoder_read() {
    return (int32_t)(encoder.htim->Instance->CNT);
}
```

On peut réaliser un premier teste simple en affichant sur l'écran oled la valeur renvoyée par **encoder_read** :



Fig. 3, 4 et 5 – Encodeur en fonctionnement

Ensuite, pour que la valeur de l'encodeur reste comprise entre une valeur maximale et minimale, on ajoute d'abord les deux préprocesseurs suivants dans le fichier d'entête :

```
#define SATURE_MAX 8
#define SATURE_MIN -8
```

Puis on modifie la fonction **encoder_read** pour que la valeur stockée soit remise à la valeur saturée si jamais on la dépasse :

```
int32_t encoder_read(){
    if ((int32_t)encoder.htim->Instance->CNT > SATURE_MAX){
        encoder.htim->Instance->CNT=SATURE_MAX;
    }
    else if ((int32_t)encoder.htim->Instance->CNT < SATURE_MIN){
        encoder.htim->Instance->CNT=SATURE_MIN;
    }
    return (int32_t)(encoder.htim->Instance->CNT);
}
```

Voici le code utilisé pour l'affichage sur l'écran oled en faisant attention à ne rafraichir l'écran que si la valeur a été modifiée :

```
#include "ssd1315.h"
#include "fonts.h"
#include "tim.h"
void setup(){
    encoder_init(&htim2, SATURE_MAX, SATURE_MIN);
    ssd1315_Init();
    ssd1315_Clear(SSD1315_COLOR_BLACK);
    ssd1315_Refresh();
}

#include <stdio.h>
#include <inttypes.h>
void loop(){
    static int32_t last = 0;
    int32_t pos = encoder_read();
    if (pos != last){
        char buffer[16]={0};
        snprintf(buffer, 16, "%" PRIu32 " ", pos);
        ssd1315_Draw_String(0, 0, buffer, &Font_11x18);
        ssd1315_Refresh();
        last = pos;
    }
}
```

3. partie II : le driver du moteur pas à pas

On code les fonctions **init_stepper** et **launch_stepper** suivant les consignes de l'énoncé :

```
STEPPERMOTOR steppermotor;

void init_stepper(
    TIM_HandleTypeDef * htim_pwm_p,
    uint32_t pwm_channel_number_p,
    GPIO_TypeDef * gpio_direction_p,
    uint16_t gpio_direction_pin_p,
    GPIO_TypeDef * gpio_ms1_p,
    uint16_t gpio_ms1_pin_p,
    GPIO_TypeDef * gpio_ms2_p,
    uint16_t gpio_ms2_pin_p,
    GPIO_TypeDef * gpio_enable_p,
    uint16_t gpio_enable_pin_p)
{
    steppermotor.htim_pwm           = htim_pwm_p;
    steppermotor.pwm_channel_number = pwm_channel_number_p;
    steppermotor.gpio_direction     = gpio_direction_p;
    steppermotor.gpio_direction_pin = gpio_direction_pin_p;
    steppermotor.gpio_ms1           = gpio_ms1_p;
    steppermotor.gpio_ms1_pin       = gpio_ms1_pin_p;
    steppermotor.gpio_ms2           = gpio_ms2_p;
    steppermotor.gpio_ms2_pin       = gpio_ms2_pin_p;
    steppermotor.gpio_enable        = gpio_enable_p;
    steppermotor.gpio_enable_pin    = gpio_enable_pin_p;
    steppermotor.speed              = 0;
    steppermotor.step_target        = 0;
    steppermotor.step_actual        = 0;

    HAL_GPIO_WritePin(steppermotor.gpio_ms1, steppermotor.gpio_ms1_pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(steppermotor.gpio_ms2, steppermotor.gpio_ms2_pin, _PIN_RESET);

    HAL_GPIO_WritePin(
        steppermotor.gpio_direction, steppermotor.gpio_direction_pin, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(steppermotor.gpio_enable, steppermotor.gpio_enable_pin, GPIO_PIN_SET);

    HAL_TIM_PWM_Start(steppermotor.htim_pwm, steppermotor.pwm_channel_number);
}
```

```
#define SATURE_MAX 8
#define SATURE_MIN -8
int tab_ARR[8] = {14970, 7575, 3012, 1497, 500, 250, 166, 125};

void launch_stepper(int speed){

    if(speed>SATURE_MAX){
        speed=SATURE_MAX;
    }
    else if(speed<SATURE_MIN){
        speed=SATURE_MIN;
    }

    if(speed==0){
        stepper_motor.htim_pwm->Instance->CCR1=0;
    }
    else{

        if(speed<0){
            HAL_GPIO_WritePin(
                stepper_motor.gpio_direction,
                stepper_motor.gpio_direction_pin,
                GPIO_PIN_RESET);
            speed*=(-1);
        }
        else{
            HAL_GPIO_WritePin(
                stepper_motor.gpio_direction,
                stepper_motor.gpio_direction_pin,
                GPIO_PIN_SET);
        }

        stepper_motor.speed=speed;
        stepper_motor.htim_pwm->Instance->ARR=tab_ARR[speed-1];
        stepper_motor.htim_pwm->Instance->CCR1=tab_ARR[speed-1]/2;
        HAL_GPIO_WritePin(stepper_motor.gpio_enable, stepper_motor.gpio_enable_pin,0);
    }
}
```


4. partie III : mise en lien des deux parties

On modifie légèrement les fonctions **setup** et **loop** définies précédemment pour y ajouter un appel des fonctions permettant d'initialiser et de faire tourner le moteur :

```
void setup(){
    encoder_init(&htim2, SATURE_MAX, SATURE_MIN);
    ssd1315_Init();
    ssd1315_Clear(SSD1315_COLOR_BLACK);
    ssd1315_Refresh();

    init_stepper(
        &htim3,
        TIM_CHANNEL_1,
        GPIOA,
        GPIO_PIN_7,
        GPIOA,
        GPIO_PIN_11,
        GPIOB,
        GPIO_PIN_12,
        GPIOA,
        GPIO_PIN_12);
    launch_stepper(0);
}

void loop(){
    static int32_t last = 0;
    int32_t pos = encoder_read();
    if (pos != last){
        char buffer[16]={0};
        snprintf(buffer, 16, "Speed = %" PRId32 " ", pos);
        ssd1315_Draw_String(0, 0, buffer, &Font_11x18);
        ssd1315_Refresh();
        last = pos;
    }
    launch_stepper(pos);
}
```

Il n'est pas possible d'afficher une vidéo pour voir le résultat dans ce compte rendu, toutefois le code est bien fonctionnel.

Nous avons essayé d'obtenir l'affichage du nombre de tour par minute mais le manque de temps ne nous a pas permis de le tester. Voici la fonction utilisée pour le calculer et la modification de la fonction pour l'afficher :

```
#define STEPS_PER_REV      400
#define TIM_APB1_TIMER_HZ  80000000

float get_rpm(){
    if(steppermotor.speed==0){
        return 0;
    }
    uint32_t psc = 799;
    uint32_t arr = steppermotor.htim_pwm->Instance->ARR;
    uint32_t f_tick = TIM_APB1_TIMER_HZ / (psc + 1);
    float f_step = (float)f_tick / (float)(arr + 1U);
    float rpm     = (f_step * 60.0) / (float)STEPS_PER_REV;
    GPIO_PinState dir =
        HAL_GPIO_ReadPin(steppermotor.gpio_direction, steppermotor.gpio_direction_pin);
    if (dir == GPIO_PIN_SET) {
        rpm = -rpm;
    }
    return rpm;
}
```

```
void loop(){
    static int32_t last = 0;
    int32_t pos = encoder_read();
    if (pos != last){
        char buffer1[30]={0};
        snprintf(buffer1, 30, "Speed level = %" PRIu32 " ", pos);
        ssd1315_Draw_String(0, 0, buffer1, &Font_7x10);
        char buffer2[30]={0};
        snprintf(buffer2, 30, "%f tr/min", get_rpm());
        ssd1315_Draw_String(0, 15, buffer2, &Font_7x10);
        ssd1315_Refresh();
        last = pos;
    }
    launch_stepper(pos);
}
```